LANTERN: A Lightweight Trajectory-Set Planner for Quadrotor Navigation

Abstract— **Learning-based trajectory-set planners output multiple candidate trajectories and their scores in a single forward pass, preserving several avoidance directions at once. However, existing methods cover the space ahead with fixed-length motion primitives whose endpoints do not shrink with the goal distance, so they cannot stop precisely at a goal; moreover, each candidate reads only the feature at its own feature-map location, so global context must come from a large image backbone, and performance drops sharply when the network is shrunk. We present LANTERN, a lightweight trajectory-set planner that represents each candidate as a query: a directional anchor sets the direction it covers, and cross-attention lets it read the features it needs from the whole image. Because a candidate's role no longer fixes where it reads, a narrow encoder suffices for generating and scoring candidates; and by letting a group of queries switch to a goal-relative form near the goal, the planner stops precisely at the goal. Across simulated forests, boxes, and wall corridors, LANTERN achieves higher navigation success than learning-based and optimization-based baselines, runs its forward pass 3.93 times faster than YOPO, and reaches the goal precisely in every episode under a strict stopping criterion. In real-world flights across diverse indoor and outdoor scenes, the simulation-trained policy navigates autonomously onboard without fine-tuning.**

Index Terms—Aerial systems: perception and autonomy, motion and path planning, deep learning methods, collision avoidance.

I. Introduction

High-speed quadrotor flight through forests and confined spaces requires a planner that continually updates the motion from the latest observations. Map-based planners chain online mapping, path search, and trajectory optimization [1], [2], [3] and have enabled swarm flight in the wild [4] and high-speed navigation [5], but their online computation grows with environment complexity.

Learning-based methods move this computation into offline training. Since regressing a single trajectory averages across distinct avoidance directions [6], trajectory-set methods predict multiple candidates and their costs [7]; YOPO [8] learns candidates and scores jointly in a single forward pass through a differentiable cost.

However, the performance of trajectory-set planners remains tied to backbone capacity. In YOPO, the k-th primitive both represents direction k and reads only the k-th feature-map location, so the encoder must compress the obstacle information along the path into that location; this calls for a ResNet backbone, and recent work explores even larger vision transformers [9]. Replacing the YOPO backbone with a narrow CNN reduces its dense-forest successes from 28 to 6 (Section IV-C). The bottleneck of a compact planner thus lies in the fact that a candidate's role fixes where it reads. Moreover, end-to-end policies output only velocity or acceleration commands [10], [9], and the primitive endpoints of trajectory-set methods do not shrink with the goal distance, so existing learning-based planners cannot stop precisely at a goal.

To address both problems, we present LANTERN, a lightweight trajectory-set planner (Fig. 1) whose central idea is to decouple the role of each candidate from its readout: each candidate is a query whose direction is specified by a directional anchor and whose readout region is learned from the whole image by cross-attention, so a narrow CNN only needs to extract local geometry. Once decoupled, stopping becomes just another role: a group of queries switches to a goal-relative form near the goal and shares the decoder and the scoring with the directional queries, so a single selection decides whether to avoid or to stop. Our main contributions are as follows:

1) Query-based candidate decoding. With the same narrow encoder, LANTERN succeeds in 99/100 forest episodes, whereas YOPO with per-location readout succeeds in 46/100.
2) Distance-adaptive goal-relative queries. Under a strict stopping criterion, LANTERN stops precisely in every episode, whereas neither YOPO nor the model without these queries ever does.
3) Cost-weighted set supervision built on YOPO's trajectory-cost framework, which allocates supervision by cost and separates candidate endpoints, raising dense-forest successes from 33 to 50.
4) With only 0.260M parameters, LANTERN succeeds in 196 of 200 episodes across four static scenarios, versus 124–163 for learning-based and optimization-based baselines, runs its forward pass 3.93 times faster than YOPO, and transfers zero-shot to indoor and outdoor real-world flights; code and configurations will be released.

II. Related Work

Map-based planning. Polynomial trajectories [11], [12] and constrained continuous optimization [13], [3] produce smooth, trackable motion; Fast-Planner [1] and EGO-Planner [2] replan locally on online maps, and SUPER [5] and integrated planning and control [14] couple planning with tracking for high-speed flight. Their online computation grows with map updates and iterative optimization.

Learning-based planning. Early work regressed control commands directly from images [15], and imitation of a privileged expert [7], differentiable physics [10], and reinforce-

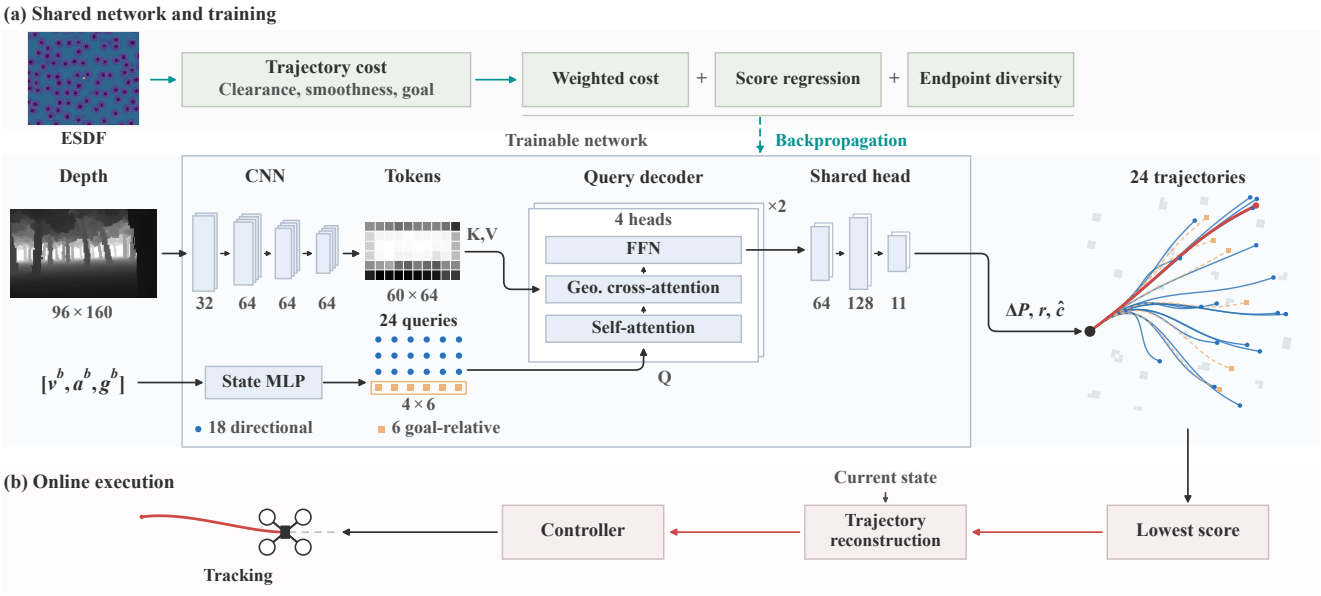


Fig. 1. Training and execution of LANTERN. (a) Shared encoding and directional queries produce 24 quintic candidates and their scores; during training, a common cost computed from the distance field supervises both the trajectories and the scores, and dashed arrows denote gradients. (b) Online, the lowest-scoring candidate is selected and rebuilt from a refreshed initial state.

ment learning [16] later enabled fast, agile flight. End-to-end policies such as DiffPhys and ViTFly [9] map depth directly to velocity or acceleration commands and fly along a desired direction without generating an explicit trajectory to a goal point; ViTFly also compares convolutional, recurrent, and vision-transformer backbones. Deep-PANTHER [17] learns spline trajectories from expert demonstrations, and YOPOv2-Tracker [18] extends YOPO to agile tracking. The closest work to LANTERN is YOPO: with a fixed initial state and horizon, both describe the same class of quintic curves, and LANTERN inherits YOPO's cost framework, score supervision, state transformation, and simulation environment; the two differ in candidate organization, feature readout, near-goal behavior, and candidate supervision (Section III).

Structured set prediction. Multiple-choice learning represents ambiguity with multiple outputs [19], [6]; MultiPath [20] and CoverNet [21] organize motion-prediction outputs with anchored hypotheses or trajectory sets; DETR [22] decodes sets with learnable queries. These methods decode predictive distributions or detection boxes, whereas each LANTERN query generates an executable trajectory from the current state, and one trajectory cost supervises both generation and selection.

III. Method

LANTERN consists of candidate generation, set training, and receding-horizon execution (Fig. 1): the network adjusts directionally anchored curves according to depth and motion state and scores them; training uses a common trajectory cost to guide both generation and selection; and online execution rebuilds the lowest-scoring candidate.

A. Problem Formulation and Trajectory Representation

At each replanning step, the input is a single depth image D_t and the motion state $s_t = [\boldsymbol{v}_t^b, \boldsymbol{a}_t^b, \boldsymbol{g}_t^b] \in \mathbb{R}^9$, whose three components are the velocity, the acceleration, and the displacement to the goal. The network outputs $K = 24$ candidates and their scores, and the lowest-scoring candidate becomes the tracking reference. Trajectories are generated in the body frame (forward-left-up) with the planning start as origin, and R_{wb} rotates them into the world frame (north-west-up).

Each candidate is a quintic Bézier curve with horizon T and six control points $P_0^k, \dots, P_5^k$. Because a quintic polynomial is uniquely determined by the position, velocity, and acceleration at both ends [23], the first three control points follow directly from the current state:

$$P_0^k = \mathbf{0}, \qquad P_1^k = \tfrac{T}{5}\boldsymbol{v}_t^b, \\ P_2^k = \tfrac{2T}{5}\boldsymbol{v}_t^b + \tfrac{T^2}{20}\boldsymbol{a}_t^b. \tag{1}$$

All candidates are therefore consistent with the current motion state, and the network only needs to determine the last three control points P_3^k, P_4^k, P_5^k and the score $\hat{c}_k$, parameterized as described in Section III-C. The selected curve is executed by a geometric tracking controller [24].

B. Shared Depth Encoding and Directional Queries

Unlike dense prediction, which binds each candidate's role to its readout position, LANTERN decouples the two: the anchor sets the direction a candidate covers and cross-attention sets where it reads, so the encoder only extracts local obstacle geometry and a single encoding serves all 24 candidates.

The encoder is a four-layer narrow CNN that encodes a 96×160 depth image into 6×10 image tokens of 64

dimensions; the normalized motion state is embedded by an MLP. The 24 queries form a 4×6 grid that uniformly covers $\pm 45^\circ$ in yaw and $\pm 30^\circ$ in pitch. Each query fuses the motion state, its directional geometry, and a learnable identity, so it knows both the current motion condition and the direction it covers.

The shared decoder consists of two Transformer decoder layers [25]: each layer first applies self-attention among the queries and then cross-attention to obstacle features of the whole image, and the cross-attention adds a fixed bias, determined by the angular difference between queries and image tokens, as a spatial prior. A shared output head predicts 11 quantities per query: residuals $\boldsymbol{z}_3^k, \boldsymbol{z}_4^k, \boldsymbol{z}_5^k$ for the three control points (9 values), a travel-distance adjustment, and a score; a lower score indicates a lower predicted cost. The whole model has only 0.260M parameters, more of them in the decoder than in the encoder, so the budget is spent mainly on candidate reasoning rather than visual encoding.

C. Directionally Anchored Candidate Generation

When control points are regressed directly, the candidates of a compact network tend to converge to a single solution (Section IV-C). LANTERN therefore assigns each candidate a nominal motion along a directional anchor, and the network predicts only a bounded adjustment to it:

$$\bar{P}_i^{k,\text{nav}} = f_i r_k \boldsymbol{d}_k + R_k(\beta_{\text{nav}} f_i r_k \tanh \boldsymbol{z}_i^k), \quad i = 3, 4, 5. \tag{2}$$

Here, $\boldsymbol{d}_k$ is the anchor direction, r_k the travel distance adjusted by the network, $f_i = i/5$, R_k rotates the anchor frame into the body frame, $\boldsymbol{z}_i^k$ is the predicted residual, and β_{nav} sets the adjustment magnitude. The first term places each candidate along its own direction, the second term reshapes the curve according to the observation, and tanh keeps the adjustment close to the nominal motion.

D. Distance-Adaptive Goal-Relative Queries

Fixed-length anchors cannot stop at the goal. In LANTERN, the last six queries therefore form an auxiliary subset $\mathcal{T}$ whose form adapts to the goal distance d:

$$\begin{aligned} \bar{P}_i^k &= (1-q)\,\bar{P}_i^{k,\text{nav}} + q\big[\boldsymbol{g} + \beta_{\text{term}} \min(d, R_{\text{on}}) \tanh \boldsymbol{z}_i^k\big], \\ q &= \text{clip}\left((R_{\text{on}} - d)/(R_{\text{on}} - R_{\text{full}}), 0, 1\right), \quad k \in \mathcal{T}. \end{aligned} \tag{3}$$

Far from the goal, $q = 0$ and the auxiliary queries take part in obstacle avoidance like the other directional queries. Within R_{on}, they gradually turn into a goal-relative form centered on the goal, with residuals shrinking linearly with d, so that candidate endpoints can land on the goal and the vehicle can stop at low speed. The remaining eighteen queries always keep their role of directional avoidance, and both kinds of queries are selected jointly by their scores within the same set. The settling term C_k^{term} in the training cost supervises deceleration and stopping for near-goal candidates.

For all 24 queries, the raw control points yield the terminal state $\boldsymbol{p}_T = \bar{P}_5$, $\boldsymbol{v}_T = 5(\bar{P}_5 - \bar{P}_4)/T$, and $\boldsymbol{a}_T = 20(\bar{P}_5 - 2\bar{P}_4 + \bar{P}_3)/T^2$. After $\boldsymbol{v}_T$ and $\boldsymbol{a}_T$ are clipped to $v_{\max}$ and $a_{\max}$, the terminal and initial states are substituted into the boundary conditions to obtain the final P_3^k, P_4^k, P_5^k analytically. Both the training cost and online execution act on this rebuilt curve.

E. Cost-Weighted Set Supervision

If trajectories and scores are trained with different objectives, the scores may drift away from execution quality. Following YOPO [8], LANTERN supervises both with the same differentiable cost; the cost of each rebuilt curve is

$$\begin{aligned} C_k = \lambda_J J_k + \lambda_A A_k + \lambda_G G_k + C_k^{\text{obs}} \\ + C_k^{\text{prog}} + \lambda_{\text{term}} \sigma_k C_k^{\text{term}}, \end{aligned} \tag{4}$$

where J_k and A_k are the integrated squared jerk and acceleration, G_k guides the curve toward the goal, C_k^{prog} and C_k^{term} encourage far-field progress and near-goal settling, respectively, and σ_k reduces the settling weight when the obstacle cost is high, leaving room for detours.

The obstacle cost should capture both sustained proximity and brief hazards. With N points sampled uniformly along each curve and h_{ki} denoting the obstacle distance at the i-th point,

$$\begin{aligned} C_k^{\text{obs}} = \frac{1}{N} \sum_{i=1}^{N} \exp\left(\frac{d_{\text{prox}} - h_{ki}}{s_{\text{prox}}}\right) \\ + \lambda_{\text{event}} \max_i \left[\frac{d_{\text{event}} - h_{ki}}{s_{\text{event}}}\right]_+^2, \end{aligned} \tag{5}$$

where $[x]_+ = \max(x, 0)$. The first term measures the average proximity along the curve; the second penalizes only the most severe violation below the threshold d_{event}, so brief hazards are not diluted by averaging. Distances are read from the paired ESDF through differentiable interpolation, and gradients flow back to the control points through the sample positions.

Given the costs of all candidates, the supervision received by each candidate is allocated according to its cost:

$$\pi_k = \text{sg}\left[(1-\varepsilon)\,\text{softmax}(-\vartheta^{-1}\boldsymbol{C})_k + \frac{\varepsilon}{K}\right]. \tag{6}$$

Unlike YOPO, which applies cost gradients to each candidate independently, this allocation gives low-cost candidates more supervision so that they converge to high-quality solutions, while the uniform component with ratio ε keeps the remaining candidates improving, balancing quality and coverage at the set level; sg denotes stop-gradient. To keep candidate endpoints from coinciding, a separation penalty is applied to the directional candidates,

$$\mathcal{L}_{\text{div}} = \frac{1}{|\mathcal{P}|} \sum_{(k,j)\in\mathcal{P}} \left[m_{\text{div}} - \|\boldsymbol{p}_T^k - \boldsymbol{p}_T^j\|\right]_+^2, \tag{7}$$

where $\mathcal{P}$ contains the ordered pairs of distinct candidates, excluding pairs within the auxiliary subset. The per-sample training objective is

$$\begin{aligned} \mathcal{L} = \sum_k \pi_k C_k + \lambda_{\text{div}} \mathcal{L}_{\text{div}} \\ + \frac{1}{K} \sum_k S_{\delta_s}(\hat{c}_k - \text{sg}[C_k]), \end{aligned} \tag{8}$$

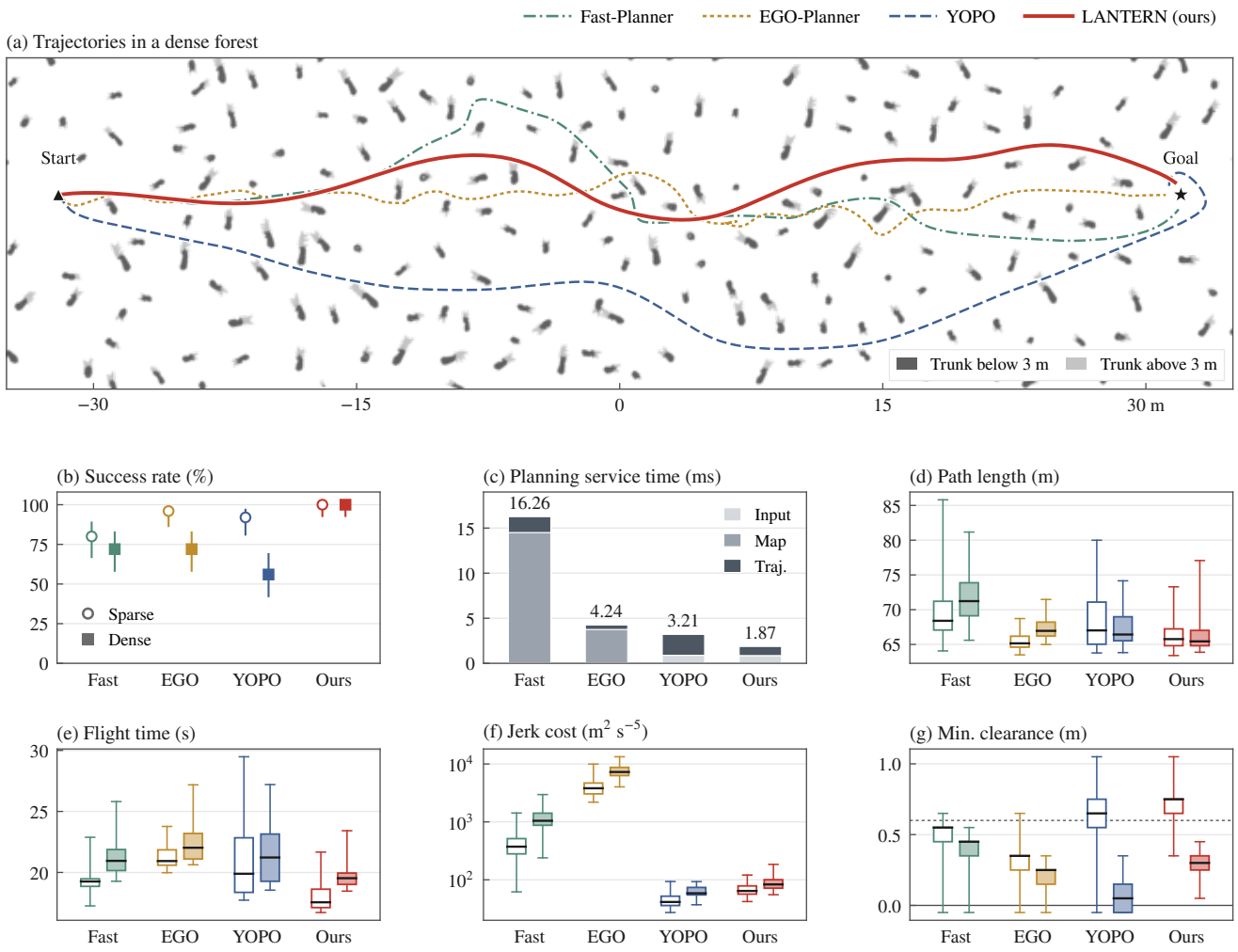


Fig. 2. Forest navigation. (a) A dense-forest map on which all four methods succeed. (b) Success rates with 95% Wilson intervals; hollow and filled markers denote sparse and dense forests. (c) Planning service time including mapping. (d)–(g) Path length, flight time, integrated squared jerk of the commanded trajectory, and minimum clearance; the first three use successful episodes, and the dashed line marks the 0.6 m near-miss threshold.

whose three terms improve the candidates, separate their endpoints, and learn the scores, with S_{δ_s} the SmoothL1 loss. The same cost thus determines both the training weights and the score targets.

F. Online Replanning and Reference Handoff

Because the vehicle keeps moving during inference, executing the curve computed at inference time would make consecutive references jump at the handoff. LANTERN therefore records the inference state x_{inf} and attitude R_{inf}, obtains the current initial state $x_h = (\boldsymbol{p}_h, \boldsymbol{v}_h, \boldsymbol{a}_h)$ from a state selector $\mathcal{S}_m$ before publishing, keeps the terminal state of the selected candidate, and analytically rebuilds a new curve with $\boldsymbol{r}_{\text{new}}(0) = \boldsymbol{p}_h$, $\dot{\boldsymbol{r}}_{\text{new}}(0) = \boldsymbol{v}_h$, and $\ddot{\boldsymbol{r}}_{\text{new}}(0) = \boldsymbol{a}_h$. When the tracking error is within 0.5 m, the initial state is taken from the previous reference at the handoff, so consecutive references are C^2 continuous; otherwise, the reference is re-initialized from the measured state. When the test speed differs from the training speed, the horizon and motion scales are rescaled by the speed ratio.

IV. Experiments

A. Implementation Details and Experimental Setup

Implementation details. The training set contains 66,000 depth images from 15 maps covering forests, walls, and other obstacles, and two held-out maps are used for validation; the test maps are generated from separate seeds and overlap with neither. Motion states and goals are sampled randomly, with 40% of the goals within 5 m. The network is trained for 120 epochs with AdamW [26] (batch size 64, learning rate 7.5×10^{-5}), with a training horizon of 2 s, speed and acceleration limits of $5\,\text{m}\,\text{s}^{-1}$ and $5\,\text{m}\,\text{s}^{-2}$, $R_{\text{on}} = 5$ m, $R_{\text{full}} = 2$ m, $\varepsilon = 0.1$, $\vartheta = 1$, and $m_{\text{div}} = 1.5$ m.

Baselines and scenarios. We compare against the learning-based trajectory-set planner YOPO [8] (official ResNet-18 configuration) and the optimization-based planners EGO-Planner [2] and Fast-Planner [1], both built on online mapping. All methods perceive depth rendered by

TABLE I

MULTI-SCENARIO NAVIGATION RESULTS, 50 EPISODES PER ROW. TIME AND PATH LENGTH ARE MEANS±STANDARD DEVIATIONS OVER SUCCESSFUL EPISODES, AND JERK IS THE MEAN RMS ACTUAL JERK OVER SUCCESSFUL EPISODES. CLEARANCE COLUMNS GIVE THE MEDIAN MINIMUM CLEARANCE AND THE NUMBERS OF EPISODES WHOSE MINIMUM CLEARANCE FALLS BELOW 0.3 AND 0.6 m (THE LATTER ARE NEAR MISSES), BOTH INCLUDING COLLISIONS.

	Outcome			Efficiency		Minimum clearance			Jerk ($\mathrm{m\,s^{-3}}$)
Method	Succ.↑	Coll.↓	Timeout↓	Time (s)↓	Path (m)↓	Median (m)↑	<0.3 m↓	<0.6 m↓	RMS↓
Sparse forest (tree spacing 4 m)									
LANTERN	**50**	**0**	**0**	$\mathbf{17.87 \pm 1.02}$	66.26 ± 2.13	**0.75**	**0**	**11**	2.00
YOPO	46	1	3	21.00 ± 3.36	68.58 ± 4.48	0.65	6	21	**1.48**
EGO-Planner	48	2	**0**	21.29 ± 0.95	$\mathbf{65.58 \pm 1.34}$	0.35	21	49	13.82
Fast-Planner	40	9	1	19.37 ± 1.12	69.61 ± 4.49	0.55	10	45	5.09
Dense forest (tree spacing 2 m)									
LANTERN	**50**	**0**	**0**	$\mathbf{19.66 \pm 0.95}$	$\mathbf{66.19 \pm 2.36}$	0.30	25	**50**	2.08
YOPO	28	22	**0**	21.50 ± 2.54	67.16 ± 2.42	0.05	48	**50**	**1.66**
EGO-Planner	36	8	6	22.58 ± 1.85	67.28 ± 1.59	0.25	41	**50**	17.84
Fast-Planner	36	12	2	21.23 ± 1.50	71.74 ± 3.71	**0.45**	**12**	**50**	7.33
Boxes (300 obstacles)									
LANTERN	**47**	2	**1**	21.91 ± 4.26	75.73 ± 11.18	**0.45**	12	39	2.56
YOPO	25	**0**	25	30.78 ± 4.72	83.91 ± 7.03	**0.45**	**9**	**33**	**1.79**
EGO-Planner	32	6	12	23.72 ± 3.42	$\mathbf{69.16 \pm 5.11}$	0.25	33	50	15.88
Fast-Planner	34	13	3	$\mathbf{21.49 \pm 2.22}$	73.01 ± 4.98	0.35	18	48	5.43
Wall corridors (80 walls)									
LANTERN	**49**	1	**0**	19.44 ± 3.58	72.14 ± 8.06	0.75	3	14	2.27
YOPO	25	**0**	25	26.76 ± 5.72	80.61 ± 10.88	**0.95**	**1**	**8**	**1.60**
EGO-Planner	47	2	1	21.24 ± 1.43	$\mathbf{65.51 \pm 2.32}$	0.45	9	47	10.05
Fast-Planner	38	10	2	$\mathbf{19.19 \pm 1.44}$	68.83 ± 4.03	0.55	11	28	4.62

the YOPO simulator and fly a 64 m route through sparse and dense forests (tree spacings 4 and 2 m), boxes, and wall corridors, with the same 50 fixed seeds per scenario; LANTERN uses a single set of weights throughout.

All methods fly at $4\,\mathrm{m\,s^{-1}}$ with an acceleration limit of $3.2\,\mathrm{m\,s^{-2}}$. An episode succeeds if the vehicle enters the 1 m goal region within 40 s without collision. Clearance is the minimum obstacle distance minus the body radius; clearances below 0.3 and 0.6 m count as low clearance and near misses, both including collisions.

B. Multi-Scenario Navigation

Forests. LANTERN completes all 100 forest episodes without collision and with the lowest mean flight time, whereas YOPO succeeds in only 28 dense-forest episodes (Table I and Fig. 2). With 2 m tree spacing every method has near misses, but the 5th-percentile clearance of LANTERN remains 0.09 m, while collisions drive it negative for all other methods. YOPO moves most smoothly, and the jerk of LANTERN is far below that of the optimization-based methods.

Boxes and walls. Among boxes and walls, LANTERN succeeds in 47 and 49 episodes, for 196/200 across the four scenarios, significantly more than the 124, 163, and 148 of YOPO, EGO-Planner, and Fast-Planner (McNemar's tests paired by seed, $p < 10^{-7}$). YOPO does not collide in these two scenarios; all of its failures are timeouts: in 37 of them it comes within 2 m of the goal yet oscillates there, flying about 40 m extra without converging, because its candidate endpoints do not shrink with the goal distance (Section III-D). The four failures of LANTERN stem from a collision during the takeoff detour, a collision after drifting off the goal altitude, and near-goal non-convergence.

C. Query Decoding and Encoder Capacity

We first verify that the gap under a narrow encoder stems from the readout (Table II(a)). Replacing the YOPO backbone with the same narrow CNN as LANTERN in the official code (YOPO-S) reduces its dense-forest successes from 28 to 6; conversely, replacing the narrow CNN of LANTERN with ResNet-18 [27], which enlarges the whole network about 44-fold, brings no improvement. Per-location readout thus depends on encoder capacity, whereas query decoding does not: with the same narrow encoder, they succeed in 99/100 and 46/100 forest episodes, respectively ($p < 10^{-15}$). To rule out other differences between the two code bases, we change only the readout within the LANTERN network to direction-aligned per-location readout, which collides in every dense-forest episode, whereas a query-decoding model trained for the same 50 epochs succeeds in 19/50 ($p < 10^{-5}$).

Where a query reads is learned rather than prescribed: removing the angular bias of the cross-attention or changing its direction keeps forest success at or above 99/100, as does removing self-attention. Fig. 3 contrasts the two readouts.

Directional anchors. Once the readout is decoupled, candidate roles can only come from the anchors. To isolate the candidate representation, this study disables cost weighting and endpoint separation. When the anchors are removed

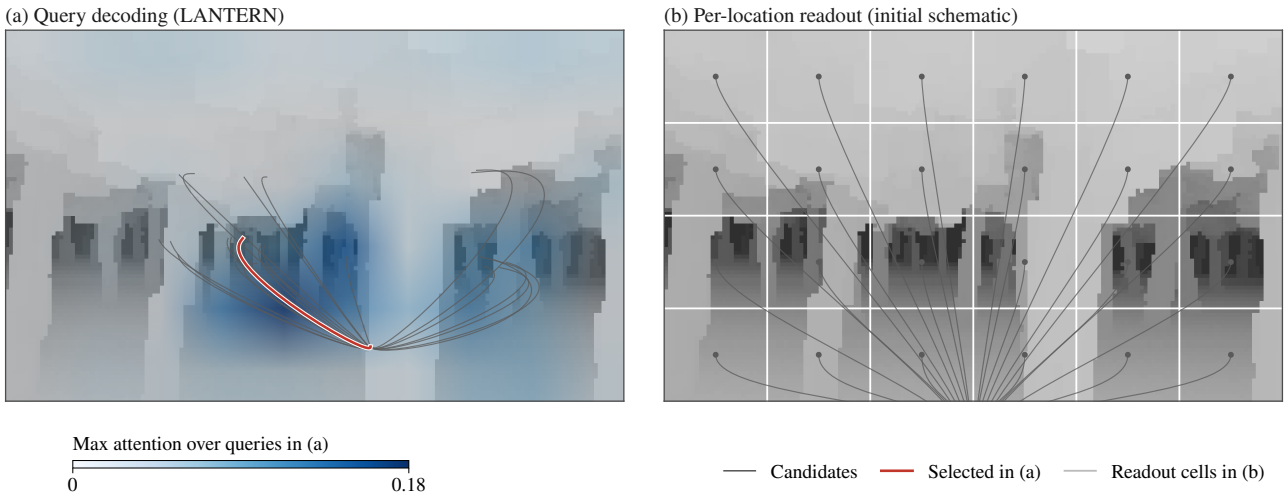


Fig. 3. Two feature readouts. (a) Actual predictions of LANTERN on a validation depth image: cross-attention averaged over two layers and four heads, then maximized over the 24 queries per location; gray: candidates, red: selected trajectory. (b) Schematic of the initial per-location layout: white lines form the 4×6 readout grid, and each fixed cell corresponds to one initial candidate endpoint (gray dot); the curves only illustrate the cell-to-candidate correspondence and are not model predictions.

TABLE II

CAPACITY, READOUT, AND ANCHOR STUDIES. (A) SUCCESSES IN THE TWO FORESTS (50 EPISODES EACH); LANTERN AND YOPO-S ARE INDEPENDENTLY RETRAINED, AND YOPO IS THE SAME AS IN TABLE I. (B) CANDIDATE REPRESENTATIONS UNDER QUERY DECODING IN THE DENSE FOREST (50 EPISODES), TRAINED FOR 50 EPOCHS WITHOUT COST WEIGHTING OR ENDPOINT SEPARATION; REGRET IS THE NORMALIZED COST GAP BETWEEN THE SELECTED AND BEST CANDIDATES. (C) TRAINING OBJECTIVES WITH THE SAME ARCHITECTURE (ONE TRAINING SEED).

(a) Readout	Encoder	Params (M)	Sparse	Dense
Query (LANTERN)	Narrow CNN	0.260	**50**	49
Query (LANTERN)	ResNet-18	11.374	**50**	**50**
Per-location (YOPO-S)	Narrow CNN	0.181	40	6
Per-location (YOPO)	ResNet-18	11.295	46	28

(b) Candidates	Coll.↓	Err. (m)↓	Spearman↑	Regret↓
Free	9	37.44	0.00	0.486
Anchored	**5**	**3.17**	**0.87**	**0.049**

(c) Objective	Sparse	Dense	Dense coll.↓
YOPO objective	49	33	16
Cost-weighted (ours)	**50**	**50**	**0**

under query decoding and control points are output directly (Table II(b)), the 24 candidates collapse into one cluster, their scores are nearly uncorrelated with the true cost, and the endpoint error reaches 37 m; with directional anchors, the error drops to 3.2 m and the score correlation rises to 0.87. Anchors specify roles and cross-attention learns the readout; both are required.

D. Precise Arrival

The main benchmark does not require stopping. To evaluate the goal-relative queries, we adopt a strict stopping criterion (error $\leq$0.1 m and speed $\leq$0.2 m s^{-1}, held for 0.5 s) and run 50 episodes per variant in forests with 4 m spacing at 5 m s^{-1}. The ablation fixes the blending weight of the auxiliary queries to zero at inference, which leaves all outputs unchanged beyond 5 m from the goal.

TABLE III

STRICT STOPPING (TREE SPACING 4 m, 50 EPISODES PER ROW). SA: STRICT STOP; NS: ENTRY INTO THE 1 m GOAL REGION; e_f, v_f: MEDIAN FINAL ERROR AND SPEED.

Method	SA↑	NS↑	Coll.↓	e_f (m)↓	v_f (m s^{-1})↓
LANTERN	**50**	**50**	**0**	**0.050**	**0.070**
w/o goal queries	0	27	20	3.347	2.037
YOPO	0	45	**0**	1.558	1.361

As Table III shows, LANTERN stops strictly in every episode with a final error of about 5 cm. Without the goal-relative queries, no episode stops, the vehicle oscillates near the goal, and 20 episodes end in collisions; YOPO enters the goal region in 45 episodes but keeps a final speed of about 1.4 m s^{-1} and never stops. Precise stopping thus comes entirely from the goal-relative queries.

E. Cost-Weighted Set Supervision

With the architecture and training pipeline fixed, we replace only the training objective with YOPO's native one (Table II(c)): both perform similarly in the sparse forest, but in the dense forest the YOPO objective succeeds in only 33 episodes, whereas cost-weighted supervision succeeds in all 50 ($p < 10^{-4}$).

F. Computational Cost

On an Orin NX with identical CUDA Graph settings, the forward passes of LANTERN and YOPO take 1.19 and 4.67 ms (3.93 times faster), or 1.68 and 5.24 ms including output readback and candidate selection. Replaying real flight

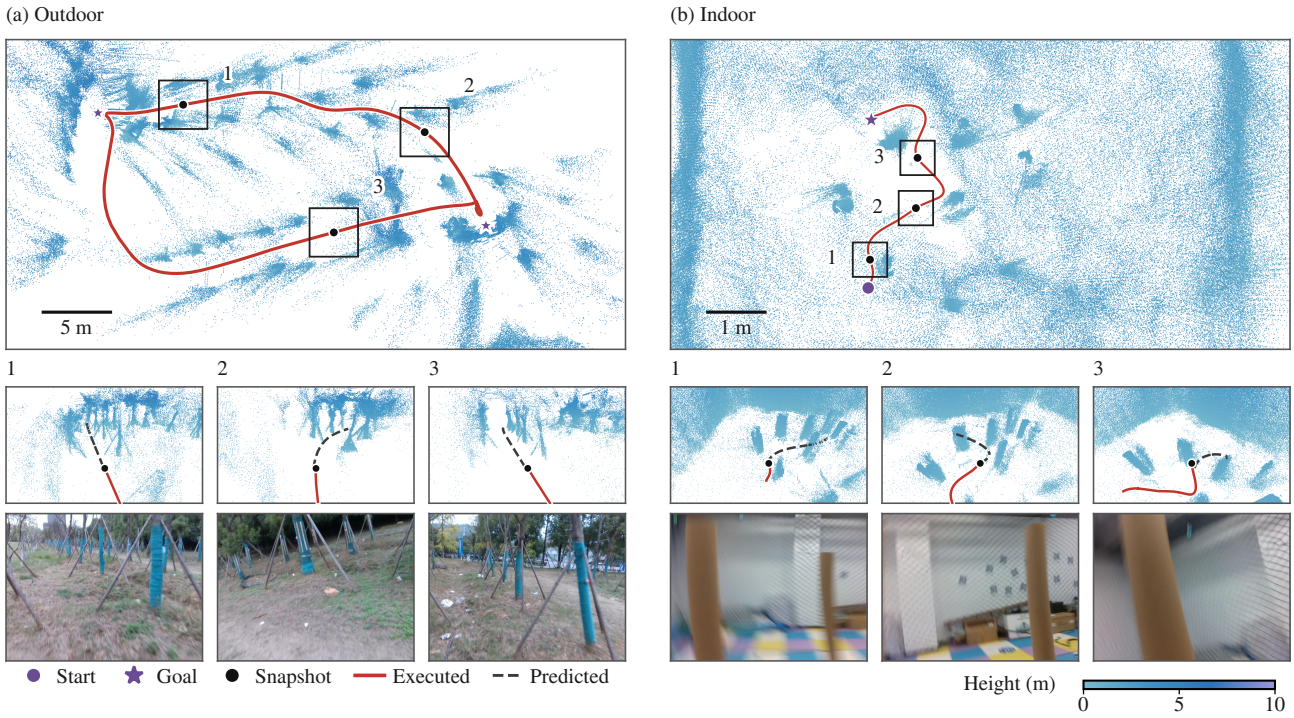


Fig. 4. Real-world flights: (a) outdoor flight; (b) indoor flight through pillars. Red: measured trajectory; dark dashed: predicted trajectory; purple dot and star: start and goal.

data on the same platform, EGO-Planner needs 15.2 ms for mapping and replanning alone.

G. Real-World Deployment

The platform is a 520 g quadrotor carrying a RealSense D435 depth camera and a Jetson Orin NX, localized by motion capture indoors and by VINS-Fusion [28] outdoors. The simulation-trained policy is deployed directly without fine-tuning and replans with the depth stream at about 30 Hz.

As Fig. 4 shows, the outdoor flight reaches a peak speed of 5.13 m s^{-1}, and the indoor flight completes 15 consecutive waypoint segments with horizontal errors below 0.5 m; more flights appear in the supplementary video.

V. Conclusion and Future Work

We have presented LANTERN, a lightweight trajectory-set planner whose central idea is to decouple the role of each candidate from its readout: a directional anchor specifies the direction a candidate covers, and cross-attention lets it learn where to read in the whole image. This decoupling lets a narrow encoder suffice for generating and scoring a trajectory set, and lets stopping enter the same candidate set as a group of goal-relative queries; cost-weighted set supervision keeps the set both high-quality and well covered. In simulations across four static scenarios, LANTERN with only 0.260M parameters succeeds in 196 of 200 episodes, more than any compared method, runs its forward pass 3.93 times faster than YOPO, and stops precisely in every episode under a strict criterion, whereas YOPO never does. These results show that the key to a compact trajectory-set planner lies not in encoder capacity but in whether a candidate's role fixes where it reads.

LANTERN currently plans from a single depth frame, so it can neither distinguish static from moving obstacles nor anticipate obstacle motion. Future work will add a memory of past observations to the query decoder, estimating obstacle motion from short depth sequences and ego-motion so that each query scores its candidate against the future occupancy of obstacles and avoids moving obstacles in advance, while retaining the single forward pass and the compact network for onboard real-time performance.

References

[1] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, "Robust and efficient quadrotor trajectory generation for fast autonomous flight," *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 3529–3536, 2019.

[2] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "EGO-Planner: An ESDF-free gradient-based local planner for quadrotors," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 478–485, 2021.

[3] Z. Wang, X. Zhou, C. Xu, and F. Gao, "Geometrically constrained trajectory optimization for multicopters," *IEEE Trans. Robot.*, vol. 38, no. 5, pp. 3259–3278, 2022.

[4] X. Zhou *et al.*, "Swarm of micro flying robots in the wild," *Sci. Robot.*, vol. 7, no. 66, p. eabm5954, 2022.

[5] Y. Ren *et al.*, "Safety-assured high-speed navigation for MAVs," *Sci. Robot.*, vol. 10, no. 98, p. eado6187, 2025.

[6] C. Rupprecht *et al.*, "Learning in an uncertain world: Representing ambiguity through multiple hypotheses," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 3611–3620.

[7] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," *Sci. Robot.*, vol. 6, no. 59, p. eabg5810, 2021.

[8] J. Lu, X. Zhang, H. Shen, L. Xu, and B. Tian, "You only plan once: A learning-based one-stage planner with guidance learning," *IEEE Robot. Autom. Lett.*, vol. 9, no. 7, pp. 6083–6090, 2024.

[9] A. Bhattacharya *et al.*, "Vision transformers for end-to-end vision-based quadrotor obstacle avoidance," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2025, pp. 1–8.

[10] Y. Zhang, Y. Hu, Y. Song, D. Zou, and W. Lin, "Learning vision-based agile flight via differentiable physics," *Nat. Mach. Intell.*, vol. 7, no. 6, pp. 954–966, 2025.

[11] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2011, pp. 2520–2525.

[12] C. Richter, A. Bry, and N. Roy, "Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments," in *Robotics Research*, ser. Springer Tracts in Advanced Robotics. Cham: Springer International Publishing, 2016, pp. 649–666.

[13] F. Gao, W. Wu, Y. Lin, and S. Shen, "Online safe trajectory generation for quadrotors using fast marching method and Bernstein basis polynomial," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2018, pp. 344–351.

[14] W. Liu, Y. Ren, and F. Zhang, "Integrated planning and control for quadrotor navigation in presence of suddenly appearing objects and disturbances," *IEEE Robot. Autom. Lett.*, vol. 9, no. 1, pp. 899–906, 2024.

[15] A. Loquercio, A. I. Maqueda, C. R. del Blanco, and D. Scaramuzza, "DroNet: Learning to fly by driving," *IEEE Robot. Autom. Lett.*, vol. 3, no. 2, pp. 1088–1095, 2018.

[16] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, 2023.

[17] J. Tordesillas and J. P. How, "Deep-PANTHER: Learning-based perception-aware trajectory planner in dynamic environments," *IEEE Robot. Autom. Lett.*, vol. 8, no. 3, pp. 1399–1406, 2023.

[18] J. Lu *et al.*, "YOPOv2-Tracker: An end-to-end agile tracking and navigation framework from perception to action," arXiv preprint arXiv:2505.06923, 2025.

[19] A. Guzmán-Rivera, D. Batra, and P. Kohli, "Multiple choice learning: Learning to produce multiple structured outputs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 1799–1807.

[20] Y. Chai, B. Sapp, M. Bansal, and D. Anguelov, "MultiPath: Multiple probabilistic anchor trajectory hypotheses for behavior prediction," in *Proc. Conf. Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 100, 2020, pp. 86–99.

[21] T. Phan-Minh, E. C. Grigore, F. A. Boulton, O. Beijbom, and E. M. Wolff, "CoverNet: Multimodal behavior prediction using trajectory sets," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 14 062–14 071.

[22] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. European Conf. Computer Vision (ECCV)*, 2020, pp. 213–229.

[23] M. W. Mueller, M. Hehn, and R. D'Andrea, "A computationally efficient motion primitive for quadrocopter trajectory generation," *IEEE Trans. Robot.*, vol. 31, no. 6, pp. 1294–1310, 2015.

[24] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)," in *Proc. 49th IEEE Conf. Decision and Control (CDC)*, 2010, pp. 5420–5425.

[25] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[26] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7

[27] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[28] T. Qin, S. Cao, J. Pan, P. Li, and S. Shen, "VINS-Fusion," Software, 2019, accessed: Nov. 5, 2026. [Online]. Available: https://github.com/HKUST-Aerial-Robotics/VINS-Fusion